

面向 AI Agent 系统效应的语义火焰图

AgentSight 研究原型

2026 年 6 月 14 日

摘要

AI 编程 agent 的困难不只是“它说了什么”，而是它把用户意图转化成了哪些真实系统效应：进程、文件、网络、测试、失败重试和 token 消耗。现有 LLM observability 工具擅长展示单次运行的 trace tree，传统 profiler 擅长展示 CPU 或内存热点，但二者都难以回答一个用户更常问的问题：哪些任务语义导致了最多、最绕或最重复的系统行为？本文提出一种语义火焰图模型：用本地小模型给 session、prompt 和 LLM call 生成一个词标签；让 tool/process/file/network 足迹通过确定性 lineage 或 agent-native tool log 继承标签；再把路径折叠成 flamegraph stacks。在本机一个 AgentSight 仓库的 36 个 Codex/Claude 真实 session 上，agent-native 原型将 4031 个 tool event 展开为 5312 个系统观测并折叠成 2270 条语义栈，压缩 2.34 倍；去掉 session/prompt 语义后，392 个 nonsemantic buckets 会混合多个语义区域，覆盖 68.505% 的观测权重，flat effect baseline 的混合权重达到 74.473%。这些结果说明语义帧能在该 workload 中分开普通进程摘要会合并的行为区域。本文也明确边界：用户效用实验和 live exact AgentSight capture 尚未完成；当前 C6 只在 4 个 effect 的 fixture 上验证 checker 接口。

1 问题：agent 的系统效应不可读

一个 agent 运行结束后，用户通常不想从第一条消息回放到最后一条消息。他们想知道：这次运行哪里最重，哪里绕路，哪里反复试错，哪些文件或网络目标被碰过，哪类 prompt 导致了最多系统动作。传统工具给出的答案都只覆盖一半。进程工具能告诉我们 git、cargo、sed 或 gh 跑过；LLM trace 工具能告诉我们模型调用、tool call 和 token；普通 flame graph 能把采样栈按宽度聚合。但是用户真正关心的是跨层关系：“为了修测试，agent 反复运行了哪些命令并读取了哪些路径”，或者“为了写论文，Claude-heavy 的 gh 行为集中在哪些 prompt 区域”。

本文的立场是：agent observability 的第一对象不应该是单个 span，也不应该是单个进程，而应该是 task-

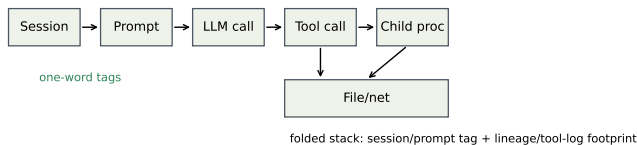


图 1: 语义标签只命名上层上下文；系统足迹不由 LLM 猜测，而是通过 exact lineage 或 agent-native tool log 继承标签并折叠成 stack。

grounded system effect。一个面向审计的图必须同时满足三点：

- 上层可读：分析者能看到 session/prompt/subagent/LLM call 对应的任务语义。
- 下层可证：exact stream 中的系统效应应能追溯到 tool call、进程、文件或网络事件；agent-native prototype 至少保留 tool/log 足迹。
- 横向可聚合：重复行为必须折叠成带权重的 stack，而不是画成越来越长的 trace tree。

这个目标与现有 LLM observability 平台互补。LangSmith、Langfuse、Phoenix、Weave、Helicone 等工具已经证明了 LLM trace 的价值：它们记录模型调用、tool calls、retrieval、latency、cost 和 nested spans。SigNoz/Inkeep 甚至已经把 agent spans 画成 flamegraph，用宽度表示 duration。这些系统解决的是“单次运行如何发生”。本文试图解决的是另一个问题：跨 session、跨 agent、跨 prompt，哪些语义区域造成了哪些系统足迹。

2 核心模型

图 1 展示了本文的模型。语义层非常小：每个 session、prompt 和 LLM call 只生成一个 lowercase word，例如 paper、debug、flamegraph。这些词不是 ontology，也不是固定分类集合；它们只是给折叠栈提供一个短、稳定、可搜索的任务帧。

系统层不交给模型判断。对于 agent-native 历史，原型从 tool call 中抽取 tool 类别、命令 basename、effect 类别、路径或域名组。对于 AgentSight exact stream，目标输入是：

```
tool_call -> shell -> child process ->
file/network effect
```

每个下层事件必须通过 tool call id、进程实例、child-process family 和时间窗口连接到 prompt/session。PID-only match 不足够，因为 live tracing 中 PID 会复用。因此系统效应继承语义标签，而不是由 LLM 对文件或网络事件重新分类。

最终 system footprint stack 为：

```
project;agent;session;prompt;tool;cmd
effect;path/domain/status
```

token footprint stack 为：

```
project;agent;session;prompt;llm;model
kind
```

火焰图的宽度是折叠后的权重。如果同一路径出现 75 次，folded file 中只有一行带权重的 stack。这是本文与 trace tree 的根本区别：trace tree 保留时序展开，semantic flamegraph 保留可聚合的 task-effect 结构。

2.1 为什么只要一个词

本文刻意不让小模型输出长摘要、层级分类或自然语言解释。原因有三点。第一，火焰图帧必须短；长标签会让最宽区域不可读，也会把视觉空间浪费在语言表述差异上。第二，标签只服务于定位和聚合，不承担判决功能。如果 paper 与 writing 被分成两个词，最坏结果是相邻区域没有合并；但如果让模型判断某次文件写入“是否合理”，错误会直接变成安全或正确性结论。第三，一个词标签可以本地化、缓存化和可替换。在当前原型中，小模型只处理 session/prompt/LLM-call 文本一次，系统事件完全通过 lineage 继承标签。这使得昂贵或隐私敏感的 prompt 处理停留在语义边界内，后续 file/network/process 聚合可以用普通程序完成。

这种设计也给论文 claim 设了边界。语义标签不是 ground truth task ontology，而是 lossy navigation frame。因此 evaluation 不问“标签是否代表真实意图”，而先问一个更可验证的问题：加入这些帧后，普通工具会合并掉的系统行为是否被分开。这正是 baseline mixing 指标要测量的内容。

3 实现

原型位于 docs/visexp。一键入口为：

```
python3 docs/visexp/run_pipeline.py --out
docs/visexp/out
```

该 pipeline 读取本机 Codex 与 Claude JSONL session，运行语义标注、folded stack 生成、SVG/论文图生成、tag stability smoke、C5 task packet 生成、C6 fixture lineage checker、artifact evaluator、visual summary、verifier 和 unit tests。在当前机器上，完整 pipeline 8 步耗时 86.606 秒，其中 semantic flamegraphs 36.657 秒，tag stability 49.702 秒。

3.1 小模型标签

标签器优先调用 llama.cpp。默认发现到 qwen2.5-3b-instruct-q4_k_m.gguf 时，对最多前 60 个 uncached 请求使用本地小模型，temperature 为 0，并用 grammar 约束输出一个词。非法输出会被丢弃并回退到 deterministic regex tagger。为了避免 fallback 缓存污染模型结果，cache key 包含 tagger namespace，即 fallback 或模型 basename。

当前运行中共有 3243 个 tag 请求，其中 3213 个命中缓存，30 个本地小模型 uncached 调用全部成功，fallback 使用为 0。这证明小模型路径是可运行的，但不能证明语义充分；C2 只支持 one-word grammar 和 provenance，C7 的 adequacy 仍需要人工标签。

3.2 多维投影

除了完整 system/token flamegraph，原型还输出五个投影：session-system、prompt-system、session-token、prompt-token、llm-token。投影不改变总权重，只删除一个或多个语义轴。例如 session-system 删除 prompt 帧，用来观察同一 session 下系统行为是否跨 prompt 聚合；prompt-system 删除 session 帧，用来观察同名 prompt tag 是否在不同 session 中共享系统足迹。verifier 强制检查这些投影的总权重必须与原始 system/token 总量一致。

3.3 隐私与复现

提交 artifact 不包含原始 prompt。prompt-tags.csv 中 prompt preview 默认 redacted，只保留 hash、tag 和计

数。`verify_artifacts.py` 检查 prompt redaction、folded totals、manifest hash、dimension projection totals 和敏感字符串。`input-manifest.json` 记录 argv、session content hash、脚本 hash、模型 checksum 和 repo dirty 状态。

3.4 设计取舍

实现中最重要的取舍是把“解释”与“关联”分开。LLM 只解释上层文本上下文；确定性代码负责解析工具调用、展开命令、折叠路径、生成火焰图和校验总量。这避免了一个常见陷阱：让 LLM 读完整 trace 后输出一个看似合理的故事。那样的故事很容易吸引人，却无法证明每个文件、网络或测试动作来自哪次 tool call。本文需要的是可审计 profile，因此每条 folded stack 都必须能回到一个输入事件集合。

第二个取舍是保留多个视图而不是寻找唯一正确图。`session-system` 适合做高层审计，`prompt-system` 适合定位导致重系统行为的 prompt 类型，`llm-token` 适合看模型调用的 token accounting。这些图共享同一套 folded-stack grammar 和 verifier，所以不是三个互不相干的 dashboard，而是同一事实表上的不同投影。如果某个投影的总权重与源图不一致，verifier 会失败。这让可视化从“漂亮截图”变成可测试 artifact。

第三个取舍是承认 agent-native history 和 exact system stream 的差距。Codex/Claude JSONL 能较好恢复 tool、命令和部分路径域名，但它不是内核级审计日志。AgentSight 的独特点在于已经能把 `tool_call` $\rightarrow$ `shell` $\rightarrow$ `child process` $\rightarrow$ `file/network effect` 精确连起来。因此本文把 agent-native flamegraph 作为可复现实验输入，把 AgentSight-shaped checker 作为 exact lineage 接口；下一步要用 live snapshot 把二者合并。

4 评估

4.1 评估原则

我们把 claim 分成三类。第一类是 artifact-internal claim: folded totals 是否一致、prompt 是否被 redacted、维度投影是否守恒、标签是否满足一个词 grammar。这类 claim 可以由 verifier 和 unit tests 完成。第二类是 partitioning claim: 语义帧是否分离了 baseline 会混合的行为。这类 claim 由 nonsemantic mixing 和 flat mixing

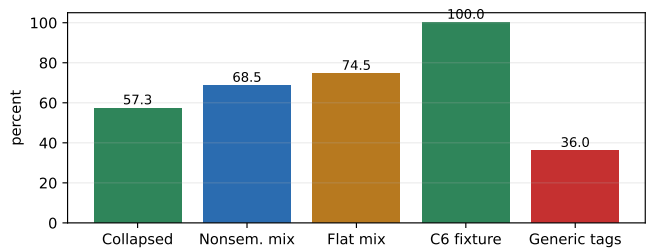


图 2: 当前 artifact 的主要结果。Collapsed 表示语义 system stack 的折叠观测占比；Nonsem. mix 和 Flat mix 表示去掉语义帧后 baseline bucket 混合多个 prompt/session 的观测权重占比。

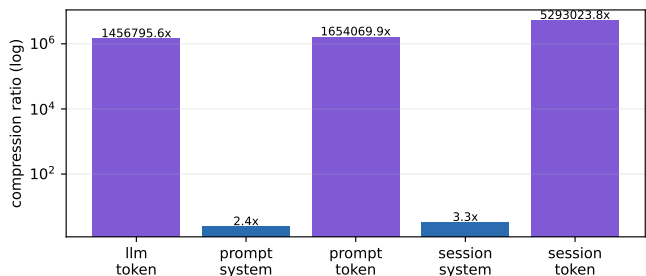


图 3: 不同语义维度投影的压缩率。Token 视图因为按 token 权重聚合，压缩率远高于 system event 视图；这些数字只能作为 source-local accounting。

度量支撑。第三类是 user/system claim: 用户是否更快回答问题，以及 live AgentSight capture 是否完整连接 effect lineage。这类 claim 目前不能靠本地 artifact 自证，所以在本文中必须保守呈现。

这种 gate 化写法使得负结果不会污染正结果。即使 C5 用户实验尚未完成，C1–C3 仍然可以成立：它们只依赖本机真实 session、折叠栈守恒和 baseline 对比。反过来，如果后续用户实验不显著，semantic flamegraph 仍可能是一个好的 forensic profile，只是不能宣称提升普通用户效率。

4.2 RQ1: 语义帧是否分割 baseline 混合？

表 1 和图 2 给出当前本机结果。36 个 session 包含 4031 个 raw tool event。路径和域名展开后得到 5312 个 system observations，折叠成 2270 条 unique semantic stacks，压缩率为 2.34 倍。更关键的是 baseline mixing: 如果去掉 session/prompt 帧，392 个 nonsemantic buckets 会混合多个语义区域，覆盖 68.505% 的观测权重；如果进一步退化 flat effect bucket，397 个 buckets 混合多个语义区域，覆盖 74.473% 的观测权重。这个指标是 proxy:

表 1: 当前 artifact 指标。

指标	数值
Sessions	36
Raw tool events	4031
Expanded system observations	5312
Unique semantic system stacks	2270
Collapsed observations	3042
Semantic compression	2.34x
Unique prompt tags	38
Generic prompt rows	35.987%
Nonsemantic mixed weight	68.505%
Flat mixed weight	74.473%

它证明语义帧在该 workload 中分割了 baseline 会合并的 buckets，但不证明用户效用或标签语义正确。

这个结果直接回应“传统工具也能看到”的质疑，但只在 artifact-level 成立。传统 flat summary 可以告诉我们 `git read` 或 `cargo test` 很重，但不能告诉这些重行为分别来自 `paper`、`debug`、`flamegraph` 还是 `session` prompt 区域。混合 bucket 的高权重说明语义帧不是装饰，而是在该 workload 的 baseline 中被合并掉的信息。

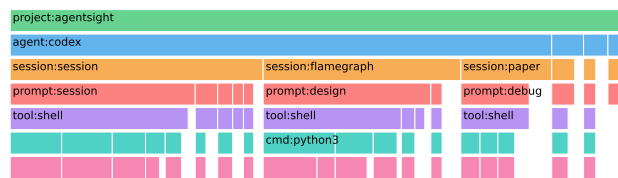
4.3 Case study: 本仓库 session 能看出什么

表 2 展示了当前运行中最宽的一组真实 stack。这些 stack 不包含原始 prompt，却保留了足够的任务上下文。例如，传统 process summary 会说 Codex 的 `git read` 出现 376 次；语义 stack 进一步显示，这个 bucket 实际混合了 27 个 `session/prompt` 区域。最大几个区域分别是 `session/session`、`flamegraph/design`、`paper/debug`、`flamegraph/branch` 和 `session/trace`。另一个例子是 `cargo test`：82 次测试行为被 baseline 合并，但语义帧把它分成 `session` 解析、`paper debug`、`model` 检查和 `SSL/debug` 等区域。

这个例子也说明本文不能只画一棵 trace tree。trace tree 能重放某次 `git` 调用的位置，但不能自然回答“跨多个 prompt，哪些 `git read` 属于同一调试或设计阶段”。flat process summary 可以聚合 `git read`，但会丢掉任务区域。semantic flamegraph 把二者连起来：宽度来自重复系统行为，层级来自 `session/prompt/tag` lineage。因此它适合做三类 artifact audit：重区域，例如 `session/session` 的 `git read` 行为；绕路区域，例如同一个 prompt tag 下反复测试或反复读取相同路径；重复区域，例如多个 session 都把相同 collector 路径作为读取热

表 2: 真实本机 session 的 top semantic stacks 摘要。

语义区域	系统行为	权重
Codex	<code>git read, status ok</code>	75
<code>session/session</code>		
Codex	<code>git read, status ok</code>	56
<code>flamegraph/design</code>		
Codex <code>paper/debug</code>	<code>git read, status ok</code>	38
Codex	<code>sed</code> <code>read</code>	34
<code>session/session</code>	<code>collector/src/sources</code>	
Codex	<code>python3 on semantic flame-</code>	32
<code>flamegraph/design</code>	<code>graph tests</code>	



excerpt of top 80 semantic system stacks; width = aggregated observations

图 4: 系统语义火焰图的 top-stack 摘要。完整交互式候选图在 `docs/visexp/out/system-flamegraph.svg`。

点。这些是审计 agent 工作量时会问的问题，而不是单纯的性能热点。

4.4 RQ2: 单独看 session/prompt/LLM 维度有用吗？

图 3 展示了维度投影。`session-system` 有 1603 条 unique stacks，压缩 3.314 倍；`prompt-system` 有 2170 条 unique stacks，压缩 2.448 倍。这说明 session 是更粗的聚合单位，适合回答“这次会话整体做了什么”；prompt 是更细的聚合单位，适合定位“哪类用户请求导致了某个系统足迹”。

Token 维度压缩率更高：`session-token` 压缩 5293023.833 倍，`llm-token` 压缩 1456795.55 倍。这不是性能结论，而是说明 token 是天然大权重聚合量。因此论文只把 token flamegraph 用作 source-local accounting，不做跨 agent 成本比较。

4.5 RQ3: 标签是否稳定？

当前 C7 只能给 partial verdict。语法上，prompt tag invalid count 为 0，same-hash prompt tag conflict

为 0。tag stability smoke 对 24 个 session/prompt/LLM fragments 重复 2 次, fallback 与 llama 路径在本次 run 中都没有 invalid output。但是 generic prompt row share 仍有 35.987%。这意味着 one-word tags 现在足够作为导航 hint, 但还不足以作为语义 ontology。顶会版本需要人工 adequacy labels, 并报告模型间一致性和 generic-rate 降低策略。

4.6 RQ4: exact lineage 接口是否可执行？

作为接口烟测, C6 的 checker 已经可执行。在 AgentSight-shaped fixture 中, 4 个 process/file/network effects 全部连接到 session/tool/prompt ancestry, join rate 为 100.0%, 并输出 exact-effect folded stacks。但这只是 fixture-only checker evidence, 不属于主 workload 指标。本文不能声称 live exact capture 已完成; 真正的 C6 需要对真实 AgentSight session snapshot 运行同一 checker, 并把 unjoined in-scope effect 作为 collector 或 join bug。

5 相关工作

Flame graphs. Brendan Gregg 的 flame graph 把采样栈折叠成宽度可读的性能图, 核心思想是“找最宽的块”。Grafana、Datadog、Splunk 等系统也把 profiling flamegraph 用于 CPU、内存或 profile drilldown。本文借用 folded stack 形式, 但 stack frame 不来自函数调用栈, 而来自 agent semantic context 和 system-effect lineage。

Distributed tracing. Dapper 与 OpenTelemetry 体系将请求路径表示为 spans。LLM/agent observability 平台也普遍采用 trace/span tree。LangSmith 的 runs、Langfuse 的 nested observations、Phoenix 的 OpenTelemetry/OpenInference traces、Weave 的 Calls/Traces/Threads 都能帮助开发者调试单次请求。本文的差异是把 trace 转成可聚合的 task-effect profile, 并用 baseline mixing 衡量语义帧能分割哪些 baseline buckets。

Agent observability. Helicone、Braintrust、Weave、Phoenix、LangSmith 等系统都支持 tool-call tracing、session grouping、cost/latency 和 evaluation。SigNoz/Inkeep 的 agent flamegraph 已经把多 agent workflow spans 以 duration 宽度展示, 适合发现 critical path、error cascade 和 tool overhead。本文与它们的互补点在于: 我们不只画单次运行的 span duration, 而是跨 session

聚合 prompt/session/LLM tags 与系统效应, 回答“哪些任务语义造成系统足迹”。

Context and token profiling. ContextFlame、context-profiler 等工作关注 LLM context window 和 token hotspot。本文的 token flamegraph 与其方向相近, 但系统贡献在于把 token 视图与 tool/process/file/network effect 视图放入同一个语义栈模型。

6 局限与下一步

当前版本离 OSDI 强接收还有三个缺口。第一, C5 用户效用没有真实参与者结果。我们已经生成 6 个任务、answer key、participant packets、response template 和 scorer, 但没有 participant CSV。第二, C6 没有 live exact AgentSight capture。pipeline 已支持 `--lineage-snapshot`, 但当前 artifact 只提交 fixture checker。第三, C7 标签质量还不够。35.987% generic prompt rows 会削弱导航价值, 必须通过更好的小模型、batch annotation、label repair 或人工 adequacy gate 降低。

这些局限也定义了下一轮实验。如果 4 人 pilot 发现 semantic flamegraph 在 repeated/heavy/divergent tasks 上提高准确率或降低时间, 就可以把 C5 从 unsupported 提升到 partial。如果 3 个 live AgentSight sessions 的 in-scope effects 全部 join 到 prompt/tool ancestry, C6 才能成为论文的系统贡献。如果人工 adequacy labels 显示 one-word tags 足以导航但不等于 ontology, 论文应把标签定位为 lossy navigation layer, 而不是语义真值。

7 结论

本文提出 semantic flamegraph: 用本地小模型给 agent session/prompt/LLM call 生成短语义帧, 用 lineage 或 agent-native tool log 连接系统足迹, 再用 folded stack 聚合跨 session 行为。当前本机 artifact 显示, 语义帧能分离传统 process summary 合并掉的大量行为区域: nonsemantic baseline 的 mixed weight 为 68.505%, flat baseline 为 74.473%。因此 novelty 不在“又画了一张 flamegraph”, 而在把任务语义、LLM/tool history 与可测试的系统足迹投影纳入同一个可折叠模型。当前证据足以支持 artifact-level partitioning claim, 不足以支持用户效用或 live exact capture 的强 claim。

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] Brendan Gregg. AI Flame Graphs. <https://www.brendangregg.com/blog/2024-10-29/ai-flame-graphs.html>.
- [3] Benjamin H. Sigelman et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report, 2010.
- [4] OpenTelemetry. Semantic Conventions for GenAI agent and framework spans. <https://github.com/open-telemetry/semantic-conventions-genai>.
- [5] LangChain. LangSmith Observability Concepts. <https://docs.langchain.com/langsmith/observability-concepts>.
- [6] Langfuse. Observability data model. <https://langfuse.com/docs/observability/data-model>.
- [7] Arize Phoenix. What is Phoenix? <https://arize.com/docs/phoenix>.
- [8] Weights & Biases. What is Weave? <https://docs.wandb.ai/weave/concepts/what-is-weave>.
- [9] SigNoz. How Inkeep Monitors Their AI Agent Framework with SigNoz. <https://signoz.io/blog/inkeep-ai-agent-monitoring/>.